Sequence Recognition & OCR

Week 15 — Session 1: Reading a whole string, not one digit

Practical Image Processing

Lecture 15.1 ▪ 60–70 min

The big idea

- Week 13: our CNN reads *one* digit at 97%.
- Week 14: object detection *found* the meter and its number window.
- This week: read the whole numerator — a **string** like `018426` — in one shot.
- **Today (Session 1):** why cutting the string into single digits is fragile, and the end-to-end idea that replaces it.

Where we are heading

A reader that takes the cropped meter strip and outputs the **full number** — no per-digit scissors. Session 2 builds the CRNN + CTC that make it work.

Roadmap — Session 1

From one digit to a string

The end-to-end view

In-class exercise

Wrap-up

From one digit to a string

The DocScan payoff: the whole numerator



one meter strip = one string

- DocScan does not want six separate answers — it wants the reading `018426`.
- The number of digits *varies* (5, 6, sometimes with a red decimal digit).
- Order matters: `018426` $\neq$ `018462`.

Key idea

Our unit of prediction changes: from a single class to an ordered *sequence* of classes.

The same problem shows up everywhere

- License plates, receipts, invoices, IDs, handwriting — all are *sequences* of symbols.
- In every case the count is unknown and the symbols can touch or overlap.
- "Read an ordered, variable-length string from an image" is one of the most common tasks in applied vision.

Why meters are a fair proxy

A meter numerator is the cleanest version of this task: a single line, fixed alphabet (digits), short strings. Master it here, and plates/receipts are the same recipe scaled up.

Two ways to read a string

Segment-then-classify	**End-to-end**
Cut the strip into single digits, then run the Week 13 CNN on each.	Feed the whole strip; the model outputs the whole string directly.
Needs a reliable **segmenter**.	No explicit cutting step.
Errors compound across digits.	Trained to optimise the *whole* reading.

Key idea

Today: see *why* the tidy first option breaks, then meet the second.

Approach 1: segment-then-classify



- Intuitive: we already have a great single-digit classifier.
- Each stage is simple in isolation — classic "divide and conquer".
- Everything hinges on one brittle assumption: that we can cut cleanly between digits.

Why cutting is hard

- **Touching digits**: no gap to cut on.
- **Odometer roll-over**: a wheel mid-turn shows *half* of two digits.
- Variable spacing, glare, tilt, shadow — gaps appear and vanish.
- How many digits? You must decide *that* too, before reading.



rolling 7→8

?

which class?

✕ The segmenter is the weak link

If the cut is wrong, even a perfect classifier reads garbage.

Errors compound: the multiplication trap

Suppose the segmenter is perfect but each digit is read at accuracy a. A whole 6-digit reading is correct only if *every* digit is correct.

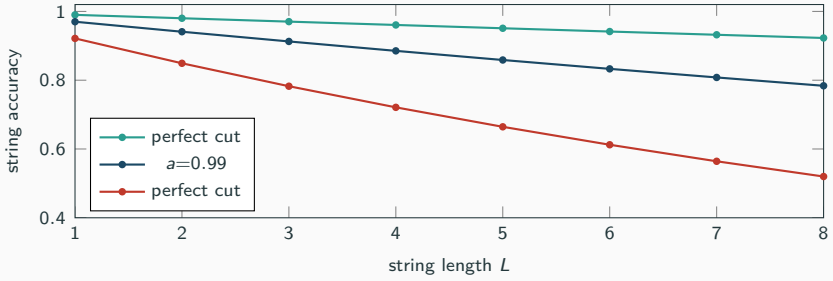
Whole-string accuracy

$P(\text{string correct}) = a^L$ for L independent digits.

per-digit a	L=1	L=4	L=6	L=8
0.97	0.97	0.885	0.833	0.783
0.99	0.99	0.961	0.941	0.923

- Our 97% digit reader becomes an **83%** *string* reader at 6 digits.
- And this is the *optimistic* case — it assumes segmentation never fails.

The two error sources, stacked



Real pipelines pay for *both* classification and segmentation error — the red curve falls fast.

So what do we actually want?

- Avoid committing to cut positions before we understand the image.
- Let the reader use **context**: a stroke that is ambiguous alone is clear beside its neighbours.
- Handle a *variable* number of digits without a separate "count the digits" step.

✓ The end-to-end promise

Read the strip as a whole and output the whole string — segmentation becomes something the model does *implicitly*, if at all.

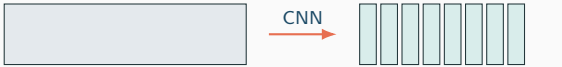
The end-to-end view

Approach 2: read the strip as a sequence



- Keep the CNN — but do *not* pool the width away. Preserve the left-to-right axis.
- The CNN turns the strip into a **sequence of feature columns**, one per horizontal slice.
- That sequence, not a single vector, is what we classify. This is the **CRNN** idea (Session 2).

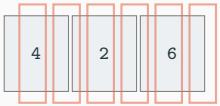
Height collapses, width survives



- Pool the *height* down to 1, but pool the *width* gently (or not at all).
- Output: T columns, each a length-C feature vector — a sequence of length T.
- Each column "looks at" a vertical slice of the strip (its **receptive field**).

A column is a soft, overlapping window

- We never cut the strip. Instead, overlapping columns tile it left to right.
- One digit may span *several* columns; a thin digit may share a column with its neighbour.
- No column has to contain exactly one digit — that is the whole point.



columns overlap; no hard cut

No “count the digits” step

- Segment-then-classify must first decide *how many* digits there are — a separate, error-prone choice.
- The end-to-end reader never counts: it emits one label per column, and the collapse rule determines the length *implicitly*.
- A 4-digit and a 6-digit strip flow through the *same* network unchanged.

✔ Good practice

Variable length is handled for free — the output length falls out of decoding, not a preset.

But now the alignment is unknown

- The CNN gives T columns (say 12); the true label has L characters (say 6). $T \neq L$.
- We do *not* know which columns belong to which digit — there are no cut marks.
- A digit spread over 3 columns will make the RNN say "4, 4, 4" — three predictions, one digit.

✕ The alignment problem

We have per-column predictions but an *unaligned*, shorter label. How do we train and decode without knowing the column-to-digit mapping?

Key idea

This is exactly what **CTC** solves — the star of Session 2.

A tiny taste of the fix (no maths yet)

- Let the model also emit a special **blank** symbol – meaning "no new character here".
- Read off the raw per-column output, then apply two rules:
  1. collapse *repeats*, then
  2. drop *blanks*.

Collapse example

raw: `4,4,-,2,2,2,-,6`

collapse repeats: `4,-,2,-,6`

drop blanks: `4,2,6`

The blank lets a repeated digit (`4,4` for *two* fours) survive — we will make this precise next time.

Why the RNN, not just the CNN?

- A CNN column sees only a local slice — good for strokes, blind to long context.
- An **RNN** passes information *along* the sequence, left to right (and right to left).
- So each column's decision can use its neighbours: "this smudge is a 1 because a 0 sits beside it."

✔ Good practice

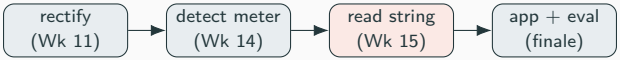
CNN = *what does this slice look like*; RNN = *what makes sense given the slices around it*.

The name: CRNN



- **CRNN** = Convolutional Recurrent Neural Network: the standard OCR backbone.
- CNN extracts, RNN contextualises, a transcription layer (CTC) turns columns into the final string.
- One network, trained end to end, from strip image to reading — no segmenter to tune.

Where this sits in DocScan



- Weeks 1–11 gave us a clean, rectified crop; Week 14 located the number window.
- This week reads it — the last *algorithmic* piece before the capstone finale.

In-class exercise

In-class exercise (10 min)

Reason about reading a whole string

1. A digit reader is 98% accurate *per digit*, cuts are perfect. What is its expected *whole-string* accuracy on a 5-digit meter?
2. Give two physical reasons a meter strip cannot be reliably cut between digits.
3. The CNN emits 10 feature columns for a 4-digit reading. Is $T > L$, $T = L$, or $T < L$ here, and why is that the comfortable case?
4. Apply "collapse repeats, then drop blanks" to the raw output `9,9,-,-,0,-,0,0`. What string results?

Debrief

- (1) $0.98^5 \approx 0.904$ — about **90%**, already a 10-point drop from the per-digit number.
- (2) Touching digits (no gap) and odometer roll-over (a wheel showing half of two digits); also glare/tilt that erase or invent gaps.
- (3) $T{=}10 > L{=}4$: more slots than characters, so blanks can absorb the slack — CTC needs $T \geq L$.
- (4) collapse `9,9`→`9`, `0,0`→`0`: `9,-,0,-,0`; drop blanks ⇒ `900`.

Key idea

You can already reason about string accuracy, why cutting fails, and the collapse rule — the CTC machinery next time makes it exact.

Wrap-up

Common pitfalls (today's ideas)

✗ Watch out

- Reporting the *per-digit* accuracy as if it were the reading accuracy.
- Trusting a segmenter on touching or rolling digits — the classic field failure.
- Pooling the *width* away in the CNN: you destroy the very axis you need to read along.
- Forgetting that T (columns) and L (characters) are different, unaligned quantities.

Good habits

✔ Good practice

- Always quote *string*-level and *character*-level accuracy, never just per-digit.
- Prefer end-to-end reading when digits touch, roll, or vary in count.
- Keep the width axis intact through the CNN; collapse only the height.
- Think in “columns”, not “cut-out digits” — overlap is a feature, not a bug.

What you can already do

- Explain why segment-then-classify multiplies errors and breaks on real meters.
- Describe the end-to-end alternative: CNN feature columns $\rightarrow$ RNN $\rightarrow$ transcription.
- State the alignment problem ($T \neq L$) and the collapse-repeats-then-drop-blanks rule.
- Place the string reader inside the full DocScan pipeline.

Next session

Session 2 — CRNN + CTC, made precise

- The CRNN in detail: CNN → map-to-sequence → BiLSTM → per-column scores.
- **CTC loss**: summing over *all* alignments so we can train without cut marks.
- Decoding: greedy (best path) vs. beam search, and the collapse rule made exact.
- Evaluating a reader with **edit distance**, CER, and whole-string accuracy — then code it and ship the meter-string reader.

Bring today's "columns, not cuts" picture — next time we teach the network to align itself.

Read the string, don’t cut it into digits.

Today: why cutting fails and the end-to-end idea. Next: CRNN + CTC make it real.